International Conference on Natural Language Processing and Computer Vision (ICNPCV)

CUDAnet: A Micro-Footprint, Dependency-Free Deep Learning Engine for Edge-Native GPU Training

Paper ID: ____

Samartha H V | Chiranth R | Pavithra N

Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education

Contents

- 1. The Problem — Race Conditions in GPU-Parallel Optimizers
- 2. CUDAnet Architecture
- 3. The Fix — Two-Phase Gradient Accumulation (2PGA)
- 4. Experimental Setup
- 5. Results — Optimizer Comparison & Training Speed
- 6. Batch Normalization Interaction
- 7. Edge AI Deployment Case
- 8. Conclusion & Future Work

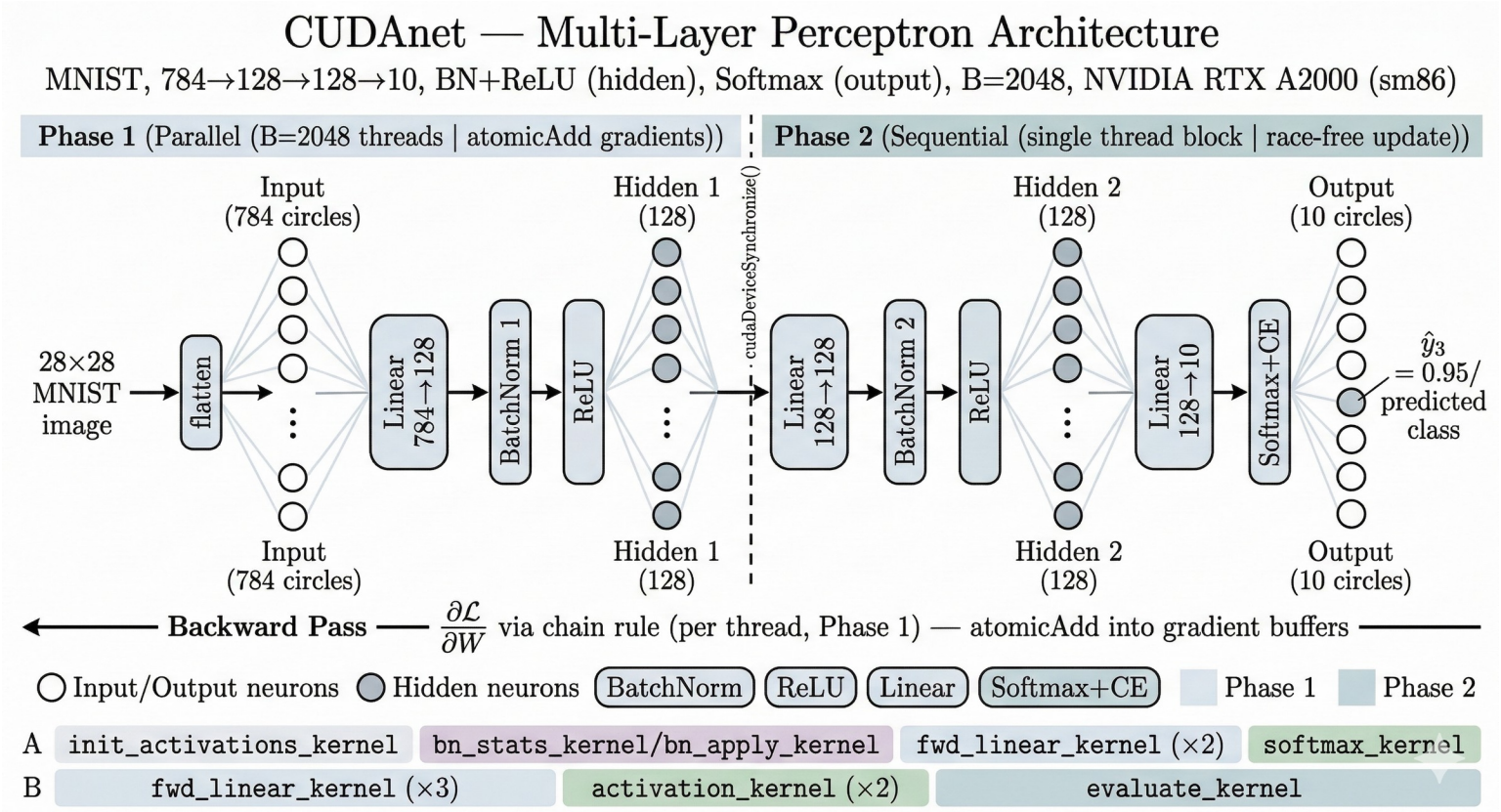
The Problem: GPU Race Conditions Break Optimizers

- Forward/backward passes across training samples are embarrassingly parallel
- But Momentum and Adam keep persistent optimizer state across steps:
  - velocity v_t (Momentum), moments m_t and v_t (Adam)
- Naive parallelization: each of B = 2,048 threads reads, updates, and writes this shared state simultaneously
- If threads i and j read v_t before either writes back — one write silently overwrites the other
- The failure is invisible: the code compiles and runs without error, and SGD (stateless) keeps working — masking the bug

Result: Momentum & Adam collapse from 97.2% → 0.0% accuracy (random chance)

CUDAnet: From-Scratch CUDA MLP Engine

- <2,300 lines of CUDA C — zero external dependencies (no PyTorch / cuBLAS / cuDNN)
- 784 → 128 → 128 → 10, BatchNorm+ReLU hidden layers, Softmax output
- Configurable layers, neurons, batch size, and 3 optimizers (SGD, Momentum, Adam)



Root Cause: Concurrent Writes to Optimizer State

Momentum update, executed by thread i:

v_t^new = β·v_t + ∇W L(i) (each thread reads-modifies-writes the SAME v_t)

- If threads i and j both read v_t before either writes back — one write is lost
- With B = 2,048 threads updating the same buffer, corruption is catastrophic and deterministic
- Adam is worse: two shared moment buffers (m_t, v_t) instead of one

Configuration	Accuracy w/ Race Condition
SGD (stateless)	Unaffected — 94.3%
Momentum	0.0% (from 97.2%)
Adam	0.0% (from 97.2%)
BN + Adam + LR	17.9% (partially masked by BN re-centering)

The Fix: Two-Phase Gradient Accumulation (2PGA)

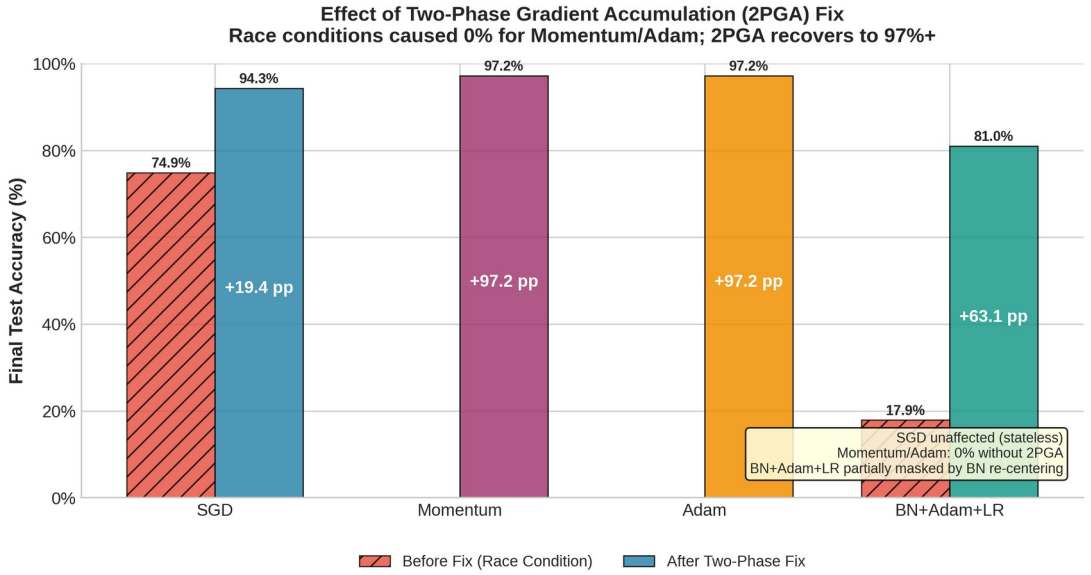
- **Phase 1 (parallel):**

  – every thread computes forward + backward pass, accumulates its gradient into a shared buffer via atomicAdd() — safe, no optimizer state touched

- **cudaDeviceSynchronize() — a global barrier**

- **Phase 2 (sequential, single thread block):**

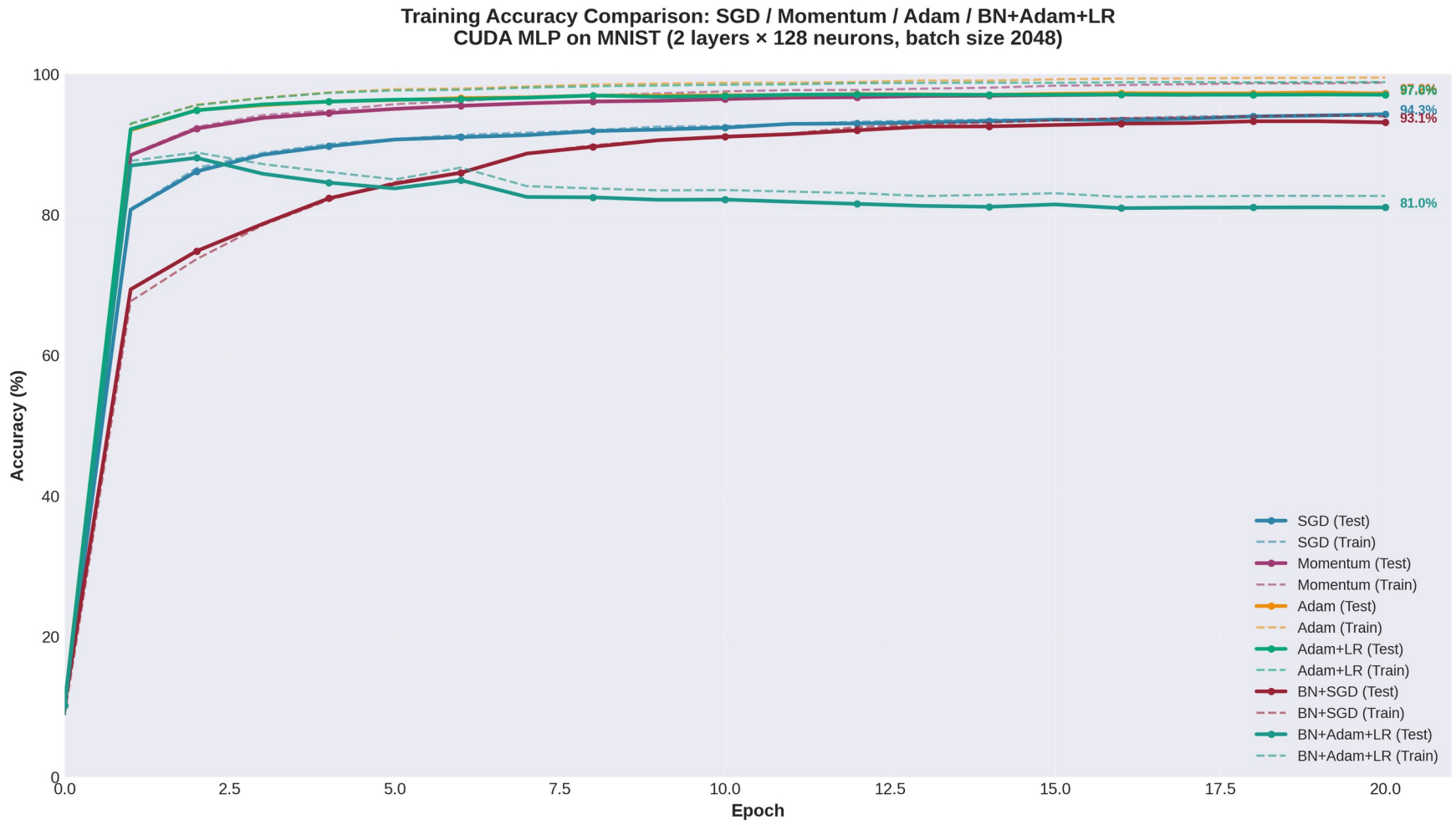
  – reads the fully-accumulated gradient once, applies the optimizer update exactly once — no concurrent writers are possible by design



Experimental Setup

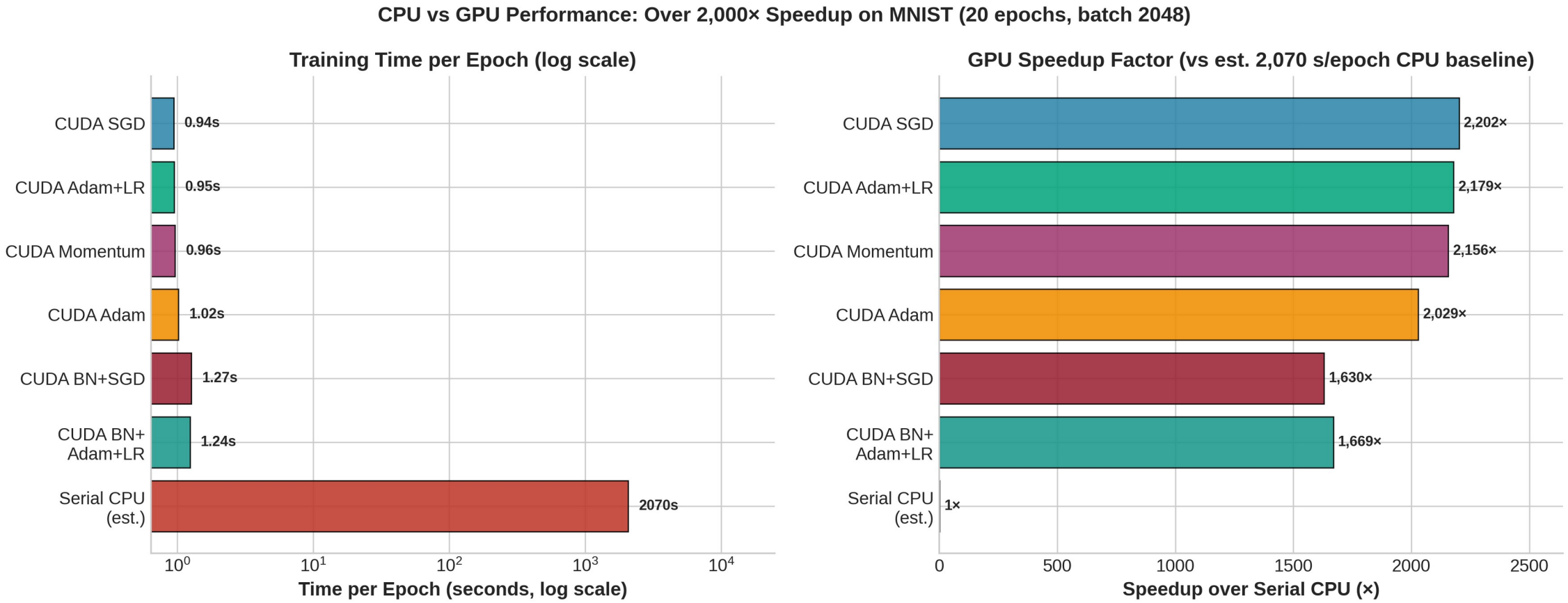
- **Hardware**
  - NVIDIA RTX A2000, 12GB (sm_86, Ampere)
  - 3,328 CUDA cores, 448 GB/s bandwidth
- **Software**
  - nvcc -arch sm_86 -O2, g++-12
  - Driver 535.288.01
- **Dataset**
  - MNIST: 60k train / 10k test, 28×28
  - 80/20 split → 48k train / 12k val, 23 mini-batches/epoch
- **Model & Training**
  - 2 hidden layers × 128 neurons, 20 epochs, batch 2,048
  - LR: 0.1 (SGD/Momentum), 0.01 (Adam, cosine per-step)

Results: Optimizer Comparison



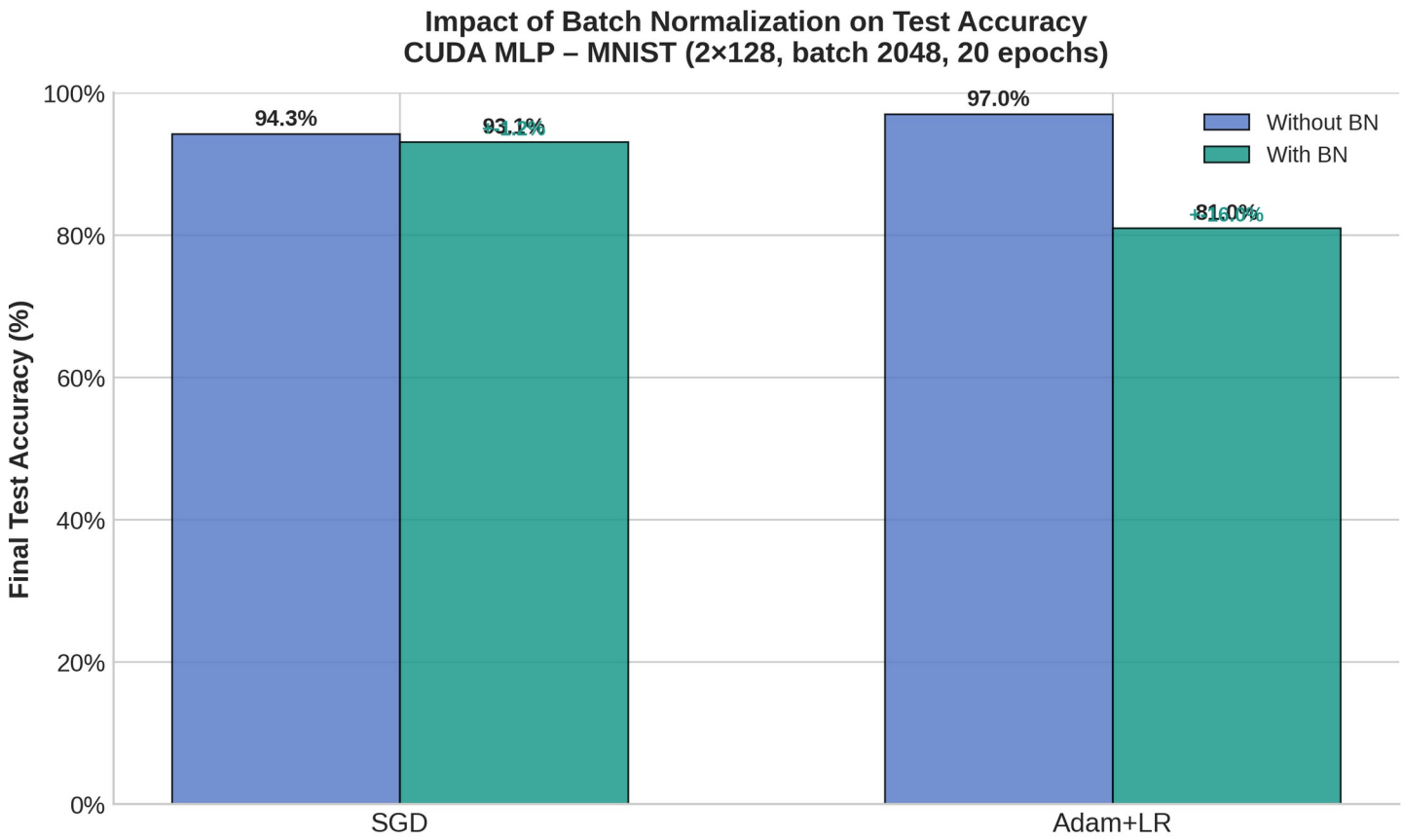
- **Momentum & Adam tie at 97.2%**
  - (+2.9 pp over SGD)
- **Neither works at all without 2PGA**
- Adam+LR (cosine): 97.0%
  - too few steps (23/epoch) for the schedule to matter much
- **BN+Adam+LR is the outlier: 81.0%**
  - peaks at epoch 2 (88.1%), then decays

Training Speed: Over 2,000× Faster than CPU



- CUDA SGD: 0.94 s/epoch → 2,202× speedup
- Heaviest config (BN+Adam+LR): 1.24 s/epoch → still 1,669×
- Serial CPU baseline: ~2,070 s/epoch (estimated, 90 s/batch × 23 batches)

Batch Normalization: A Double-Edged Sword



- **BN + SGD:**
  - −1.2 pp only (93.1% vs 94.3%) — stable at full-dataset scale
- **BN + Adam + LR:**
  - −16.0 pp (81.0% vs 97.0%)
  - peaks at epoch 2 (88.1%), then decays
- **Root cause:**
  - Adam’s moment estimates vs. BN’s shifting statistics — competing "moving targets"
  - NOT the LR schedule — per-batch cosine fix recovered only +1.1 pp

Edge AI Case: CUDAnet vs. PyTorch

Metric	CUDAnet	PyTorch
Test Accuracy (20 ep.)	97.0%	97.7%
Avg Time / Epoch	~0.95 s	0.25 s
Peak VRAM	~470 KB	248 MB (527× more)
Install Size	<2 MB	1.8 GB (900× more)
Runtime Dependencies	libcuda.so	Python + libtorch
Startup Time	<100 ms	~3–5 s

On Jetson Nano: ~0.4 ms/image inference (≈ 2,500 img/s) — clears the 30 FPS real-time threshold

Conclusion & Future Work

- **2PGA is general:**
  - any optimizer with cross-iteration state (Momentum, Adam, RMSProp, LAMB) running data-parallel on a GPU is subject to this race — 2PGA resolves it with zero staleness or locking
- **Achieved 97.2% MNIST accuracy (Momentum/Adam) at over 2,000× CPU speedup, in a <2 MB dependency-free binary**
- **Future work:**
  - Tensor Core WMMA (FP16/INT8) — expected 2–3× throughput
  - Convolutional layers via im2col + GEMM → target >99% on CIFAR-10
  - Multi-GPU scaling via NCCL AllReduce on Phase 1 gradients

Thank you — Questions?